

Q1. What is CSS Box Model?

The CSS Box Model describes how every HTML element is rendered as a rectangular box made up of four layers, from innermost to outermost:

- **Content** — the actual text, image, or other content of the element, sized by width and height.
- **Padding** — transparent space between the content and the border; increases the element's visible size.
- **Border** — a line (or none) that wraps around the padding and content.
- **Margin** — transparent space outside the border, separating the element from neighbouring elements.

By default (`box-sizing: content-box`), width/height apply only to the content area, so padding and border add to the element's total rendered size. Setting `box-sizing: border-box` makes width/height include padding and border, which is why most modern CSS resets apply it globally.

```
* {  
  box-sizing: border-box;  
}  
  
div {  
  width: 200px;  
  padding: 20px;  
  border: 5px solid #333;  
  margin: 10px;  
}
```

Q2. What is a Media Query? Explain with a sample code.

A media query is a CSS feature that applies styles conditionally based on characteristics of the device or viewport, such as width, height, orientation, or resolution. It is the core mechanism behind responsive web design, allowing a single stylesheet to adapt the layout for phones, tablets, and desktops.

```
/* Default styles (mobile-first) */  
.container {  
  display: flex;  
  flex-direction: column;  
  padding: 10px;  
}  
  
/* Tablet and above */  
@media (min-width: 768px) {  
  .container {  
    flex-direction: row;  
    padding: 20px;  
  }  
}  
  
/* Desktop and above */  
@media (min-width: 1200px) {  
  .container {
```

```

        max-width: 1140px;
        margin: 0 auto;
    }
}

```

Here the layout stacks vertically by default, switches to a horizontal row once the viewport is at least 768px wide, and gets a centred max-width container from 1200px upward.

Q3. What are the features of Bootstrap?

- **Responsive grid system** — a 12-column layout built with flexbox/grid that adapts across breakpoints (xs, sm, md, lg, xl, xxl).
- **Pre-built components** — navbars, cards, modals, carousels, accordions, dropdowns, forms, buttons, and more, ready to use.
- **Responsive utilities** — helper classes for spacing, display, flexbox alignment, text, and colour that avoid writing custom CSS.
- **Mobile-first design** — styles are written for small screens first and progressively enhanced for larger ones.
- **Cross-browser compatibility** — consistent rendering across all major browsers.
- **Customization** — Sass variables and a build system let you theme colours, fonts, and spacing.
- **JavaScript plugins** — built-in JS (no jQuery dependency from v5) for interactive components like modals and tooltips.
- **Large ecosystem** — extensive documentation, themes, and community support.

Q4. Write an example of a basic grid structure in Bootstrap.

Bootstrap's grid is based on a container, rows, and columns that always add up to 12 units per row:

```

<div class="container">
  <div class="row">
    <div class="col-md-4">Column 1</div>
    <div class="col-md-4">Column 2</div>
    <div class="col-md-4">Column 3</div>
  </div>
</div>

```

The `.container` centers content and applies responsive max-widths, the `.row` creates a horizontal flex wrapper, and each `.col-md-4` takes up 4 of the 12 columns (one-third width) on medium screens and above, stacking full-width automatically on smaller screens.

Q5. What is CSS Selectors?

A CSS selector is a pattern used to target the HTML element(s) that a rule's styles should apply to. Common types include:

- **Universal selector (*)** — selects all elements.

- **Element/type selector** (p, div) — selects all elements of that tag.
- **Class selector** (.classname) — selects all elements with that class.
- **ID selector** (#idname) — selects the single element with that ID.
- **Attribute selector** ([type="text"]) — selects elements by attribute/value.
- **Descendant/combinator selectors** (div p, div > p, h1 + p) — select elements based on their relationship to other elements.
- **Pseudo-class selectors** (:hover, :first-child) — select elements in a particular state or position.
- **Pseudo-element selectors** (::before, ::after) — style a specific part of an element.

Q6. What are CSS backgrounds? List the properties.

CSS background properties control the appearance behind an element's content and padding — colour, images, gradients, position, and repetition. Key properties:

- background-color — sets a solid background colour.
- background-image — sets one or more background images (or gradients).
- background-repeat — controls tiling (repeat, no-repeat, repeat-x, repeat-y).
- background-position — positions the image within the element.
- background-size — sizes the image (e.g. cover, contain, or explicit dimensions).
- background-attachment — controls scrolling behaviour (scroll, fixed, local).
- background-origin — defines the positioning area (border-box, padding-box, content-box).
- background-clip — defines how far the background extends (into border, padding, or content).
- background — shorthand combining all of the above.

Q7. How can we overlap elements in CSS?

Elements can be made to overlap using positioning and stacking, most commonly:

- **Positioning** — set position: relative on a parent and position: absolute on the child(ren), then use top/left/right/bottom to place them on top of each other.
- **z-index** — controls stacking order among positioned elements; a higher value renders in front.
- **Negative margins** — pulling an element with a negative margin over a neighbouring element.
- **CSS Grid** — placing multiple items in the same grid cell/area naturally overlaps them.

```
<div class="wrapper">
  
  <div class="caption">Overlapping caption</div>
</div>
```

```
<style>
.wrapper { position: relative; }
.photo { display: block; width: 100%; }
```

```
.caption {
  position: absolute;
  bottom: 10px;
  left: 10px;
  background: rgba(0,0,0,0.6);
  color: #fff;
  padding: 6px 12px;
  z-index: 2;
}
</style>
```

Q8. What are the different positioning properties in CSS?

The CSS `position` property accepts the following values:

- **static** — the default; the element follows normal document flow and top/left/right/bottom have no effect.
- **relative** — positioned relative to its own normal position; offsetting it does not affect surrounding elements' layout.
- **absolute** — removed from normal flow and positioned relative to the nearest positioned ancestor (or the initial containing block if none exists).
- **fixed** — removed from normal flow and positioned relative to the browser viewport; stays in place when scrolling.
- **sticky** — behaves like relative until a scroll threshold is reached, then behaves like fixed within its container.

Q9. What is ECMAScript?

ECMAScript (ES) is the standardized scripting language specification, maintained by ECMA International (technical committee TC39), that JavaScript is built on and conforms to. In other words, JavaScript is an implementation of the ECMAScript standard, alongside other implementations like JScript and ActionScript.

The specification defines core language features — syntax, types, statements, operators, objects, and built-in methods — while individual environments (browsers, Node.js) add extra APIs (DOM, fetch, file system, etc.) on top of it. Major versions are released yearly, known as ES2015 (ES6), ES2016 (ES7), and so on; ES6 was a landmark release introducing `let/const`, arrow functions, classes, promises, template literals, and modules.

Q10. Difference between var, let and const in JavaScript. Explain the Temporal Dead Zone.

var

- Function-scoped (or globally scoped), not block-scoped.
- Can be re-declared and updated freely.

- Hoisted to the top of its scope and initialized with undefined, so it can be accessed (as undefined) before its declaration line.

let

- Block-scoped — confined to the nearest { }.
- Can be updated but not re-declared in the same scope.
- Hoisted, but not initialized — accessing it before declaration throws a ReferenceError (see TDZ below).

const

- Block-scoped, like let.
- Must be initialized at declaration and cannot be reassigned afterward. Note: for objects/arrays, the binding is constant but the contents can still be mutated.
- Also subject to the Temporal Dead Zone.

Comparison Table

Feature	var	let	const
Scope	Function	Block	Block
Re-declaration	Allowed	Not allowed	Not allowed
Re-assignment	Allowed	Allowed	Not allowed
Hoisting behaviour	Initialized as undefined	Hoisted, in TDZ	Hoisted, in TDZ

Temporal Dead Zone (TDZ)

The Temporal Dead Zone is the span of code between the start of a block and the point where a let or const variable is actually declared. During this span the variable exists (it has been hoisted) but is not yet initialized, so any attempt to read or write it throws a ReferenceError. This differs from var, which is usable (as undefined) even before its declaration line.

```
console.log(a); // undefined (var is hoisted and initialized)
var a = 10;

console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 20;      // <-- TDZ for b ends here

{
  // TDZ for c starts at the top of this block
  console.log(c); // ReferenceError
  const c = 30;   // TDZ for c ends here
}
```

The TDZ exists to catch bugs early by preventing use of a variable before its intended declaration, encouraging a clearer, top-to-bottom declare-then-use pattern.